What Should an Inspection Drone Fleet Learn? Tool Selection, Answer Fusion and Federated Reinforcement Learning under Time Budgets

Mahdi Khashan

[a][Affiliation to be added],

Abstract

An inspection drone can ask several perception tools about a frame: a fast onboard classifier, onboard vision-language models (VLMs), or a large VLM in the cloud over a weak link. Agentic designs let a VLM decide which tool to call, and a fleet could train such behaviour with federated deep reinforcement learning. We compare this with classical alternatives on 13,729 power-line component items from ElecVQA-Bench (InsPLAD), with four tools (ResNet-50, laya-vision, SmolVLM-256M on board; Qwen3-VL-8B in the cloud), measured tool latencies, LTE uplink traces, and a risk of 10 per missed defect, 1 per false alarm and 0.5 per deferral to the operator. We formulate tool selection as a Markov decision process with a time budget per item, and as a constrained one over 40-item mission segments with a shared time budget, a deferral cap and persistent outages. A VLM used as the agent gets worse with its tools under every PPO configuration we tried. The gains come from classical steps: a logistic fusion of the tools' answers with a label-free EM threshold lowers the risk from 177.0 for the best single tool to 107.1 per 1,000 items, and deferring 7–11% of uncertain items lowers it further to 92–94 in 0.25 s per item. PPO, central and federated over five drones with 28–30 KB updates, learns budget-aware policies that match these rules but does not beat them: from scratch it is 10–25 points worse on mission segments, and warm-started by imitating a tuned rule it ties the rule, winning clearly in one of eight settings (102.8 against 106.0). The best policies answer an item in 0.18–0.25 s with 1.6–3.4 kB of upload, against 1.4–6.4 s and 25 kB per segment measured for SmolVLM-based EVLM. A federated routing controller under one-sided feedback and federated fine-tuning of the perception model (0.4–41 GB per run) complete the comparison of what a fleet can learn per transmitted byte.

Keywords: drone fleets, federated learning, reinforcement learning, tool selection, vision-language model, edge AI, infrastructure inspection

1. Introduction

Infrastructure inspection is moving from drones that record video for later analysis to fleets that interpret what they see on board [1]. For each frame, a drone can ask perception tools of very different cost: a fast onboard classifier, a slower onboard VLM, or a larger VLM in the cloud reached over a link that is often weak or down. Farahani et al. run a fine-tuned SmolVLM-256M on a Jetson in 1.4–6.4 s per segment [2] and profile whether such a model should run on board, at the edge or split [3]; both name an adaptive runtime orchestrator as future work.

Two designs for such an orchestrator are popular. In the agentic design, a VLM is the agent: it looks at the frame, decides to zoom or to ask another model, reads the result and writes the report. Reinforcement learning can train this behaviour, and a fleet can share it through federated learning. In the routing design, a small policy only decides which tool to call next; the report is computed from the tools' answers. Both designs promise budget-aware behaviour, and both are cheap to federate compared with the perception models themselves.

This paper tests these designs on cached tool outputs for power-line inspection, with measured tool times, LTE uplink traces and a cost that weights a missed defect ten times a false alarm. We ask what a fleet should learn: the agent, the policy that selects tools, the controller that routes frames, or the perception model. Our answer is partly negative, and we report it as such.

We contribute:

- evidence that a vision-language decision model used as an agent does not profit from its tools, across six PPO configurations that train its scorer or its 8.65M-parameter head, centrally and with FedAvg (Section 4);
- a fusion of tool answers with a label-free threshold from EM prior correction [13], which lowers the test risk from 177.0 to 107.1 per 1,000 items (Section 5);
- a formulation of tool selection as an MDP with a per-item time budget and as a constrained MDP over mission segments with a shared budget and a deferral cap,

Email address: mahdikhashan1@gmail.com (Mahdi Khashan)

with the tools, latencies and constraints made explicit (Sections 3 and 6);

- a comparison of classical rules with deep RL (PPO from scratch, warm-started by imitation, anchored to the rule; central and FedAvg), which shows that deferral, not learning, gives the next gain, and that learned policies match but do not reliably beat tuned rules (Sections 7 and 8);
- latency and uplink per item against published Jetson measurements (Section 9), and a comparison with a federated routing controller and federated fine-tuning of the perception model (Sections 10 and 11).

2. Related work

Edge VLMs for inspection drones. EVLM [2] runs a LoRA-tuned SmolVLM-256M on a Jetson and cuts the uplink of a 4 s segment from 485 kB to 25 kB; [3] profiles on-board, edge and split deployment, and INAR-VL [7] routes requests between edge and cloud with a bandwidth guard. All use fixed rules. Laya [5] and laya-vision [6] answer typed questions in one forward pass with calibrated probabilities instead of generating text; we use laya-vision both as a tool and as an agent.

Routing, cascades and partial feedback. Federate the Router [18] learns language-model routers with FedAvg from offline evaluations; FedHLM [19] federates escalation thresholds. BaRP [9] trains a router from bandit feedback. Zhang et al. [8] study hierarchical inference when the probability of reaching the ground-truth oracle depends on the policy. Agarwal et al. [10] evaluate federated contextual bandits. Our routing policy differs in that it can call several tools in sequence under a time budget and the report fuses all answers.

Reinforcement and federated learning. PPO [11] with a clipped objective is our learner. FedAvg [15] averages client models weighted by data size; non-IID data degrades it [16], FedProx [17] adds a proximal term, and Flower [14] provides the simulation used for the agentic experiments. Label shift between training and deployment is corrected with the EM procedure of Saerens et al. [13].

3. Data, tools and constraints

We use the tower14 split of ElecVQA-Bench [20], built on InsPLAD power-line component crops [21]: 18 training towers (25,499 items), 10 validation towers (6,672) and 14 test towers (7,057), covering 17 component types. Each item asks whether a component has a defect, or which condition it has. The validation towers hold 19.6% defects and the test towers 15.1%, so any threshold tuned on validation is biased for test. Every tool was run once on every item, and all experiments replay these cached answers with measured per-item times.

Tools. Table 1 lists the four perception tools. ResNet always runs first; laya, SmolVLM and Qwen are the tools a policy can add. Onboard times are measured per item on an RTX 2080 Ti or GV100 and multiplied by $\kappa = 9$, our estimate of the gap to a Jetson Orin NX. Qwen runs on a cloud GPU; its cost is its measured compute time plus a round trip of 50 ms and the upload of the item's JPEG over the current link.

Network. The uplink follows 4G/LTE traces [22] replayed in mission time and scaled to three regimes. In the weak regime used throughout (10% of the trace bandwidth, six 20 s outages per hour), the median bandwidth is 3.1 Mbit/s, the link is down 4.0% of the time, and a median Qwen upload takes 85 ms; in the poor regime (2%, twenty 30 s outages per hour) it takes 0.43 s with 15.9% outage, in the good regime 8 ms. During an outage Qwen is unavailable.

Risk and constraints. An item is answered with good, with its defect, or deferred to the operator. Errors cost 10 per missed defect, 1 per false alarm and 0.5 per deferral (Section 6). Table 2 lists the constraints of each experiment.

4. A VLM as the agent

We first tested the agentic design. Laya-vision answers a typed question by scoring its options in one forward pass, so it can serve as a policy: we add the actions (zoom into one of five overlapping crops, ask ResNet, ask Qwen, report) as options and read the action from its scores. Tool results return as text in the next query. PPO trains the policy on cached frozen backbone outputs of 800 training images and is evaluated on 400 test images at budgets of 1, 3 and 6 s; FedAvg runs in Flower over five drones. The report cost counts the defect risk plus errors on the component and condition questions.

Table 3 shows the outcome. In every configuration, the report written with tools is worse than the report written without them. Separate scorers for the action and the report questions did not change this (1,645–1,765 with tools against 1,592 without). Training the decision head gave the only budget-dependent behaviour: the central policy learned to stop after 1.2 steps at 1 s and 2.0 steps at 3 s, but its reports remained worse than without tools. FedAvg learned about five times more slowly and cost 8.65 MB per drone and round in int8. A pilot with the fine-tuned laya-vision as reader gave the same picture: PPO learned to call no tool, because each tool turned more correct answers into wrong ones than the reverse.

Two further checks explain part of this. When every tool answers laya's three questions directly, fusing their answers improves component accuracy from 0.872 to 0.905 (oracle 0.925), but SmolVLM fails outside its training prompt (component accuracy 0.14) and zero-shot Qwen calls almost every image defective (false-alarm rate 0.95). And test-time zoom does not help even the classifier: over

Table 1: Perception tools. Latency per item on the replay: measured desktop-GPU median, and the time charged on the drone (onboard tools: ×9 Jetson estimate; cloud: compute plus 50 ms round trip plus upload). Risk per 1,000 test items when the tool's own answer is used alone (miss 10, false alarm 1).

Tool	Model	Params	Runs on	GPU median	Charged time (mean / p95)	Upload (median)	Risk alone
ResNet	ResNet-50 [23], fine-tuned on the training towers	24M	drone	7.3 ms	0.066 / 0.066 s	–	177.0
laya	laya-vision [6], fine-tuned (4,807 images)	201M	drone	73.5 ms	0.70 / 0.91 s	–	261.0
SmolVLM	SmolVLM-256M [4] + LoRA [24] as in EVLM	256M	drone	80.8 ms	0.73 / 0.75 s	–	184.9
Qwen	Qwen3-VL-8B, zero-shot	8B	cloud	133.5 ms	0.13 s + 0.05 s + upload	33 kB	625.8

Table 2: Constraints of the tool-selection experiments.

Constraint	Value
Time budget per item (Sec. 7)	$B \in \{0.25, 0.75, 1.5, 3\}$ s, separate for each item
Mission segment (Sec. 8)	$N = 40$ consecutive items of one tower
Segment time budget	$N\beta$ with $\beta \in \{0.1, 0.15, 0.2, 0.3\}$ s per item, shared
Deferral cap per segment	$\lfloor \delta N \rfloor$, $\delta \in \{5\%, 2.5\%\}$ (2 or 1 items)
Mandatory classifier	ResNet on every item (time reserved)
Cloud availability	only when the link is up; transfer at current bandwidth
Risk weights	miss 10, false alarm 1, deferral 0.5

Table 3: A VLM agent gets worse with tools. Report cost per 1,000 test images (lower is better; defect risk plus component and condition errors), base laya-vision in 4-bit as agent and report writer, 400 test images, seed 0, 1 s budget, round 10. Steps: mean actions per image.

Method	Report cost	Defect recall	Steps
Zero-shot	1,668	0.57	1.0
PPO trains the shared option scorer			
Report only, no tools	1,505	0.77	1.0
Central, with tools	1,715	0.50	4.9
FedAvg, with tools	1,628	0.55	6.7
PPO trains the decision head (8.65M parameters)			
Report only, no tools	1,438	0.66	1.0
Central, with tools	1,570	0.48	1.2
FedAvg, with tools	1,725	0.41	3.8

five overlapping crops, the best zoom rule raises the image-level test risk from 110.0 to 126.9; the defects ResNet misses are small source crops (median 62k against 110k pixels), so zoom adds no detail. The tools are useful, but not through a reader that has to interpret their output.

5. Fusing the tools' answers

The alternative is to combine the tools' own defect probabilities. For each item we form the logit of $P(\text{defect})$ of every called tool, a flag per tool, the task and the component type, and fit a logistic regression on the validation towers; a missing tool enters as logit 0 with flag 0, so the same model scores every subset of tools. Regularization and features are chosen by five-fold cross-validation grouped by tower.

Table 4: Fusing the tools' answers is the largest gain. Defect risk per 1,000 test items (7,057 items, 1,068 defects) with 95% bootstrap intervals. Single tools use their own argmax answer.

Method	Risk	95% CI
ResNet alone	177.0	148–206
SmolVLM alone	184.9	157–211
laya alone	261.0	225–294
Qwen alone	625.8	575–679
Fusion, threshold tuned on validation	115.8	100–133
Fusion, Bayes threshold 1/11	113.8	99–130
Fusion, Bayes after EM correction	107.1	91–124
Fusion, EM correction per region	117.5	100–137
Fusion, best threshold on test (labels)	105.4	89–124
Best single tool per item (labels)	44.8	30–60

The threshold is the difficult part. A threshold tuned on validation is too low for the test towers, which hold fewer defects. Without labels, we estimate the test defect share $\hat{\pi}$ with the EM procedure of Saerens et al. [13], rescale every probability from the validation share π_v to $\hat{\pi}$,

$$p' = \frac{p\,\hat{\pi}/\pi_v}{p\,\hat{\pi}/\pi_v + (1-p)(1-\hat{\pi})/(1-\pi_v)},$$

and apply the Bayes threshold 1/11. EM is only a mean over posteriors, so a fleet can compute it federatedly.

Fusion lowers the risk from 177.0 for the best single tool to 113.8, and the EM-corrected threshold to 107.1, within 2 points of the best threshold chosen on test labels (Table 4). EM estimates a test defect share of 0.154 against the true 0.151. Estimating it per region is worse (117.5), because small regions give unstable estimates. The remaining gap to the per-item oracle (44.8) consists mostly of false alarms: in an analysis with a less regularized fusion, 719 of 725 false alarms had at least one tool answering correctly, and 31 defects were flagged by no tool at all.

6. Tool selection as a decision problem

Fusion requires calling every tool, which costs time and uplink. We therefore let a policy choose which tools to call

for each item, under a time budget, and compare classical rules with deep reinforcement learning.

6.1. Risk, fusion and costs

Let item i have label $y_i \in \{0, 1\}$ (1: defect) and let the system answer $a \in \{\text{good}, \text{defect}, \text{defer}\}$. The risk is

$$r(a, y) = 10\,\mathbb{1}[a{=}\text{good}, y{=}1] + \mathbb{1}[a{=}\text{defect}, y{=}0] + 0.5\,\mathbb{1}[a{=}\text{defer}], \tag{1}$$

reported per 1,000 items. For a calibrated defect probability q, the expected risks of the three answers are $10q$, $1 - q$ and 0.5. Hence the report is defect if and only if $q \geq 1/11$, and deferring is optimal if and only if $\rho(q) = \min(10q, 1 - q) > 0.5$, that is, for $q \in (0.05, 0.5)$.

Let $E = \{\text{L}, \text{S}, \text{Q}\}$ be the optional tools (laya, SmolVLM, Qwen) and $C \subseteq E$ the tools called so far. Each tool k returns $p_k(i) = P_k(\text{defect} \mid i)$. The fused probability for subset C is

$$\begin{aligned} p_C(i) &= \sigma\big(w_C^\top \phi_C(i)\big), \\ \phi_C(i) &= [\,\text{logit}\, p_\text{R}(i), (\text{logit}\, p_k(i))_{k \in C}, \text{task}, \text{comp.}], \end{aligned} \tag{2}$$

a logistic regression fitted on the validation towers, either one model shared by all subsets (missing tools enter as 0 with a flag) or one model per subset. The policy is trained on out-of-fold probabilities. To correct for the unknown defect share of the deployment, $\hat{\pi}$ is estimated label-free with EM [13], and every probability is shifted from the validation share π_v:

$$q = \frac{p\,\hat{\pi}/\pi_v}{p\,\hat{\pi}/\pi_v + (1 - p)(1 - \hat{\pi})/(1 - \pi_v)}, \qquad \hat{\pi} \leftarrow \frac{1}{n}\sum_i q_i, \tag{3}$$

iterated until $\hat{\pi}$ converges. $\hat{\pi}$ is a mean over posteriors, so a fleet can compute it federatedly. The time charged for a tool on item i at mission time t is

$$\begin{aligned} c_k(i) &= \kappa\, t_k^{\text{gpu}}(i), \quad k \in \{\text{R}, \text{L}, \text{S}\}, \\ c_\text{Q}(i, t) &= t_\text{Q}(i) + t_\text{rtt} + \frac{8\, s_i}{b(t)\, 10^6}, \end{aligned} \tag{4}$$

with $\kappa = 9$, $t_\text{rtt} = 0.05\,\text{s}$, s_i the JPEG size in bytes and $b(t)$ the bandwidth in Mbit/s; $c_\text{Q} = \infty$ when $b(t) = 0$.

6.2. Separate budget per item

Each item is one episode. After ResNet, the state is $s = (C, \tau)$ with the elapsed time τ (initially c_R), the budget B and the bandwidth b, sampled from the weak trace per item. The action space is

$$\mathcal{A} = \{\text{rep}, \text{call}_\text{L}, \text{call}_\text{S}, \text{call}_\text{Q}, \text{defer}\}, \tag{5}$$

with the feasible set $\mathcal{A}(s)$: rep always, call_k if $k \notin C$ and $\tau + c_k \leq B$, and defer only in the experiments with deferral. A call moves $C \leftarrow C \cup \{k\}$, $\tau \leftarrow \tau + c_k$ with reward 0. rep or defer ends the episode with reward

$$R = -\,r(a, y) - \lambda\, \tau/B, \tag{6}$$

where for rep (report) a = defect if $q_C \geq 1/11$ and good otherwise, q_C is the EM-corrected p_C, and λ weights time against risk. The policy observes a 39-dimensional feature vector: $\text{logit}\, q_C$, $|q_C - 1/11|$, $\rho(q_C)$, $\text{logit}\, p_\text{R}$; the logit and a called-flag per optional tool; $(B - \tau)/B$, $\log B$, $\log(1 + b)$, link state; $\log(1 + c_k)$ per optional tool; image size, blur, contrast and brightness; task and component (17 types).

6.3. Mission segments with shared constraints

With a separate budget, every decision is independent, so a threshold on q_C is close to optimal. In a mission, time and operator attention are shared. A segment is $N = 40$ consecutive items $i_1, \ldots, i_N$ of one tower, processed in order, with one budget $B_\text{seg} = N\beta$ and at most $D = \lfloor \delta N \rfloor$ deferrals. The bandwidth follows the trace in mission time, $b(t_0 + T)$ with T the time used so far, so outages last over many items. The episode is a constrained MDP: maximize $\mathbb{E}[\sum_{n=1}^{N} -r(a_n, y_{i_n})]$ subject to

$$\sum_{n=1}^{N} \tau_n \leq B_\text{seg}, \qquad \sum_{n=1}^{N} \mathbb{1}[a_n = \text{defer}] \leq D, \tag{7}$$

where τ_n is the time spent on item n. Both constraints are enforced by the action mask: call_k is feasible for item n only if $\tau_n + c_k \leq B_\text{seg} - T - \sum_{m>n} c_\text{R}(i_m)$, which reserves the classifier for all remaining items, and defer only while fewer than D items were deferred. The reward is $-r(a_n, y_{i_n})$ when item n is answered ($\lambda = 0$; time is a constraint, not a cost). The state adds nine features to the per-item vector: the share of items left, the share of the budget left, $\log(1 + (B_\text{seg} - T)/(N - n + 1))$, the deferrals left (absolute and per remaining item), whether defer is feasible, the time since the link last changed, the mean ρ of the last five items, and the current ρ minus that mean.

6.4. Methods

Classical methods. All use the fused, EM-corrected q_C. ResNet only reports after the classifier. All tools that fit calls laya, SmolVLM and Qwen in this order while they fit. The uncertainty cascade calls the next tool that fits while $|q_C - 1/11| \leq 0.1$ and $|C| < 2$. With deferral, the cascade with defer calls the next tool while $\rho(q_C) > 0.5$ and defers if no tool fits. For segments, the greedy cascade does the same, spending time and deferrals as items arrive. The paced cascade allows a tool only if the time spent on the item stays below $\nu\,(B_\text{seg} - T - \text{reserve})/(N - n + 1)$ and defers only if $\rho(q_C) > \theta$. Its pace $\nu \in \{0.5, 1, 2, 4, \infty\}$ and threshold $\theta \in \{0.5, \ldots, 0.9\}$ are tuned on the validation towers per budget. The oracle chooses the subset with the lowest risk per item using the label; it is a loose bound (Section 7).

Deep reinforcement learning. The policy π_θ and the value function V_ψ are multilayer perceptrons (two hidden layers of 64 units for π_θ, one for V_ψ; 6,916–7,621 policy parameters, about 28–30 KB in fp32). Infeasible actions are

Table 5: Tool selection with a separate budget per item, 3 s budget, shared fusion, no deferral, weak network. Deep RL: mean ± standard deviation over 3 seeds. Qwen: share of items sent to the cloud.

Method	Risk	Time (s)	Qwen
Classical			
ResNet only	146.9	0.07	0
All tools that fit	105.4	1.81	0.92
Uncertainty cascade	106.3	1.23	0
Deep RL (PPO)			
Central, $\lambda = 0.01$	105.9 ± 1.5	1.26	0.58
Central, $\lambda = 0.03$	104.8 ± 3.4	1.21	0.61
FedAvg, $\lambda = 0.01$	105.7 ± 1.8	1.79	0.92
FedAvg, $\lambda = 0.03$	105.7 ± 1.8	1.31	0.53
Oracle subset (labels)	65.0	0.11	0.02

masked by setting their logits to -10^9. We train with PPO [11], maximizing

$$\mathcal{L}(\theta) = \mathbb{E}_t\Big[\min(\rho_t \hat{A}_t, \mathrm{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\hat{A}_t) + c_H\,\mathcal{H}(\pi_\theta(\cdot \mid s_t)) + \alpha \log \pi_\theta(a_t^{\mathrm{rule}} \mid s_t)\Big], \tag{8}$$

with $\rho_t = \pi_\theta(a_t \mid s_t)/\pi_{\theta_{\mathrm{old}}}(a_t \mid s_t)$, $\epsilon = 0.2$, $c_H = 0.01$, four epochs per batch and Adam. Per-item episodes use Monte-Carlo returns minus V_ψ as $\hat{A}_t$. Segments use generalized advantage estimation [12], $\hat{A}_t = \sum_{l\geq 0}(\gamma\lambda_{\mathrm{G}})^l \delta_{t+l}$, $\delta_t = r_t + \gamma V_\psi(s_{t+1}) - V_\psi(s_t)$, with $\gamma = 0.98$ and $\lambda_{\mathrm{G}} = 0.9$. The last term of (8) is zero except in the anchored variant ($\alpha = 0.5$), where a_t^{rule} is the paced rule's action in the visited state. In the warm-started variants, π_θ first imitates the paced rule by minimizing the cross-entropy to its actions on 2,000 validation segments, and V_ψ regresses the rule's discounted returns, before PPO fine-tuning with learning rate 10^{-4}.

Federated training. Five drones hold disjoint tower groups of the validation towers (or component groups, Section 7.3). In each round, every drone runs one PPO update from the global policy on its own episodes, and the server averages, $\theta \leftarrow \sum_d n_d\theta_d / \sum_d n_d$, with n_d the drone's number of items. Each drone keeps its own value function. FedAvg is computed in process; it is the same update as FedAvg in Flower [14], which we used for the agentic experiments.

7. Results: separate budget per item

7.1. Learned tool selection without deferral

PPO learns a sensible, budget-aware policy (Fig. 1, Table 5). It calls laya first, more often as the budget grows (26–60% of items at 0.75 s, 65–100% at 3 s), and at 3 s reaches the risk of calling every tool in 30% less time. FedAvg matches central training and is more stable at tight budgets: at 0.75 s, 123.4 ± 0.8 against 129.5 ± 8.7. With $\lambda = 0.2$ the policy correctly learns never to call a tool, since the time penalty exceeds the average gain per item. At 0.25 s only Qwen fits, and calling it slightly increases the risk (150.6). But the policy does not beat the cascade: in Fig. 1b, PPO, FedAvg PPO and the cascade lie on the same curve.

Table 6: Deferral to the operator is the largest gain after fusion; PPO learns it only with large batches and does not beat the rule. Separate budget per item, per-subset fusion, defer available, no cap; "defer" rows defer when $\rho(q_C) > 0.5$ after their tool calls. Risk per 1,000 test items; time and deferred share at 3 s. Deep RL: central PPO, mean ± standard deviation over 3 seeds, 30 rounds.

Method	0.75 s	3 s	Time	Deferred
Classical				
ResNet only	129.2	129.2	0.07	0
All tools that fit	113.9	107.7	1.81	0
Uncertainty cascade	112.4	104.9	1.22	0
ResNet, defer	105.5	105.5	0.07	0.135
Cascade with defer	98.5	93.9	0.25	0.067
All tools, defer	96.4	92.2	1.81	0.110
Deep RL (PPO)				
2,048 episodes per round	115.1 ± 4.7	105.8 ± 1.6	1.27	0
8,192 episodes per round	105.6 ± 4.6	95.4 ± 2.0	0.37	0.083
8,192, entropy 0.05	109.9 ± 2.3	93.8 ± 1.9	0.63	0.083
Oracle (labels, no defer)	88.3	57.8	0.10	0

The oracle (65.0) is not a reachable target. It improves on all-tools fusion on only 127 of 7,057 test items (108 false alarms, 19 misses), and on 83% of them the fused probability lies within 0.1 of the threshold, where the tool subsets fall on different sides and the label decides which subset to pick. Richer fusion does not move this floor: adding laya's full outputs and image statistics gives 118.5 with logistic regression and 151.3 with gradient boosting, against 106.3 for the simple fusion, whose AUROC is already 0.991.

7.2. Deferral

What the oracle exploits, uncertainty at the threshold, is exactly where deferring pays off: by (1), deferring costs 0.5 while reporting an item with $q \approx 1/11$ costs about 0.9 in expectation. We add defer to $\mathcal{A}$ and fit one fusion model per subset.

Deferring 7–11% of items lowers the risk by 11–15 points (Table 6); the cascade with defer reaches 93.9 at 3 s in 0.25 s per item, because it defers instead of calling slow tools. The gain also holds on the validation towers (108.8 to 93.2 for the best rule at 3 s). With 2,048 episodes per round, PPO never defers: defer costs 0.5 on easy items, where reporting costs almost nothing, and the policy suppresses it before it learns to use it selectively. With 8,192 episodes per round, it defers 8% of items and reaches 93.8–95.4 at 3 s, the level of the rules, but it is worse at 0.75 s.

7.3. Drones that differ

A fixed rule should suffer when drones differ, which is the case federated learning is meant for. We assign each of five drones one component group (glass insulators, polymer insulators, hardware, yokes, rods and grips; 795–1,912 validation items each) and its own network regime (poor, good, weak, poor, good), without deferral. We add local PPO on each drone alone, FedAvg PPO followed by 10

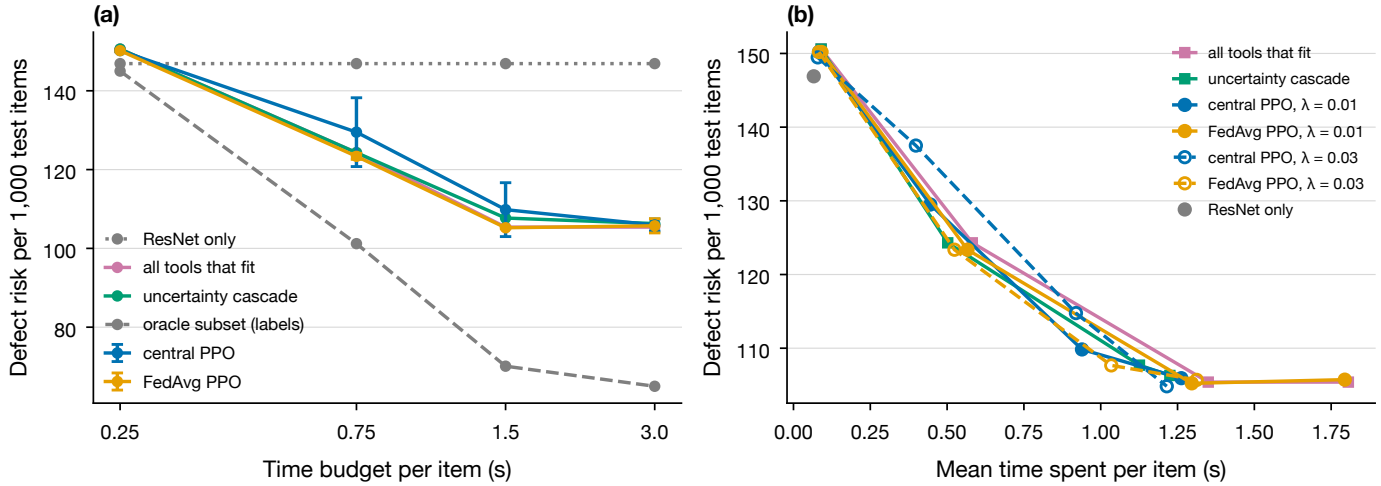


Figure 1: PPO learns budget-aware tool selection and matches calling every tool in less time, but not more accurately than a cascade. Shared fusion, no deferral, weak LTE. (a) Defect risk per 1,000 test items at each time budget; PPO lines are means over 3 seeds after 30 rounds of 2,048 episodes, whiskers one standard deviation, $\lambda = 0.01$. (b) The same methods as risk against the mean time spent per item; filled markers $\lambda = 0.01$, open markers $\lambda = 0.03$. The oracle needs 0.11 s on average and is omitted from (b).

Table 7: Drones with different component groups and network regimes: fleet risk per 1,000 test items (7,057 items). Deep RL: mean ± standard deviation over 3 seeds.

Method	0.75 s	1.5 s	3 s
Classical			
Static cascade	126.5	109.3	106.3
Cascade tuned, one rule	126.5	108.3	107.3
Cascade tuned per drone	127.4	120.2	117.1
Deep RL (PPO)			
Central	138.1 ± 8.3	116.1 ± 6.1	107.7 ± 1.4
Local (each drone alone)	131.7 ± 6.9	110.0 ± 2.0	107.1 ± 2.3
FedAvg	126.5 ± 0.0	113.0 ± 5.4	110.8 ± 4.6
FedAvg + local fine-tune	126.6 ± 0.1	112.0 ± 2.6	107.9 ± 0.9
Oracle (labels)	105.1	73.6	66.8

rounds of local fine-tuning, and cascades whose tool order, margin and maximum number of tools are tuned on validation, either one rule for the fleet or one per drone. A second split assigns tower groups. The fusion and the EM estimate are shared by the fleet; per-drone EM failed, estimating 12% defects for the hardware drone, which has 0.8%.

Heterogeneity does not change the conclusion (Fig. 2, Table 7). FedAvg with local fine-tuning is the most stable learned method and at 3 s matches the cascade (107.9 ± 0.9 against 106.3) in 1.05 s instead of 1.23 s per item, but no method lowers the risk. Tuning the cascade per drone is clearly worse (117.1), because a drone's validation and test towers differ: the glass drone sees 35% defects in validation and 20% in test. The rule tuned for the whole fleet keeps the order laya, SmolVLM, Qwen at every budget. The tower split gives the same ranking.

8. Results: mission segments

Shared constraints are where a learned policy should have an advantage: a rule that spends time and deferrals greedily cannot save them for a more uncertain item later in the segment. Table 8 and Fig. 3 report the segment experiments, all with per-subset fusion, defer and the weak network.

Constraints make rules matter. With shared constraints, pacing helps: at $\beta = 0.1$ s the paced cascade beats the greedy one by 8–9 points at both caps. At generous budgets, greedy spending is better (96.9 against 100.5 at 0.3 s and 5%), because the pace tuned on validation is too cautious on the test towers.

PPO from scratch fails. PPO without a warm start is 10–25 points worse than the best rule in every cell. Monte-Carlo returns over 40 items gave 142 in a first run; GAE brought it to 110–124. Adding the memory features (recent uncertainty, link age) changed little. At the 2.5% cap, FedAvg learned never to defer and stays near ResNet only. The policy always spends the full budget and the full deferral cap, but on the wrong items.

Warm start reaches the rule, PPO hardly refines it. Imitating the paced rule (98% agreement with its actions) brings the policy to 1–4 points of the best rule, and at $\delta = 5\%$, $\beta = 0.2$ s even 2 points below it, probably because one network trained across budgets transfers better from validation to test than separately tuned rules. Free PPO fine-tuning then degrades the copy in 14 of 16 cells, by up to 5 points. Anchored PPO changes it by at most about 2 points: it beats the best rule clearly only at $\delta = 5\%$, $\beta = 0.2$ s (102.8 ± 1.3 central and 102.9 ± 0.8 FedAvg against 106.0), ties in three cells and loses by 1–4 points

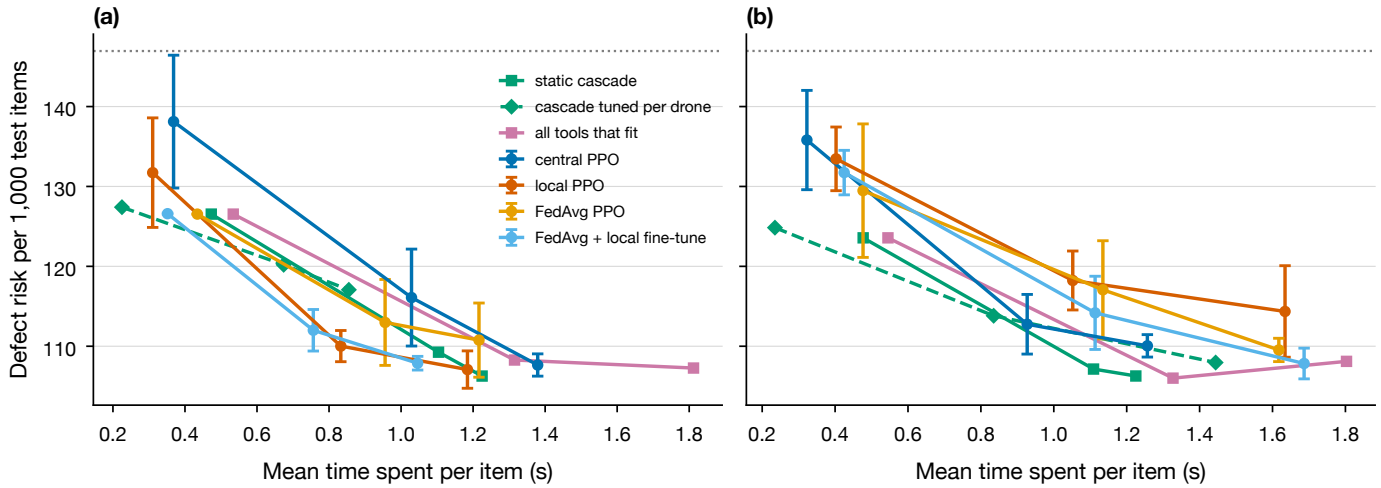


Figure 2: When drones differ in components and network, learned policies still do not beat the cascade; FedAvg with local fine-tuning is the most stable. Fleet defect risk per 1,000 test items against mean time per item, at budgets of 0.75, 1.5 and 3 s (left to right along each line), $\lambda = 0.01$. (a) Each drone inspects one component group. (b) Each drone covers one tower group. Learned methods: means over 3 seeds, whiskers one standard deviation. Dotted line: ResNet only.

Table 8: Mission segments: classical rules against deep RL. Risk per 1,000 test items (7,057 items in 40-item segments with a shared time budget and deferral cap; weak LTE in mission time). Columns: deferral cap δ and time budget per item β. Deep RL: mean ± standard deviation over seeds (3 for PPO from scratch, 5 otherwise), after 30 (scratch) or 60 (warm-started) rounds of 256 segments. Imitation only: the warm-started policy before PPO, mean over 5 seeds. Bold: best per column. Rules spend 0.09–0.24 s per item and defer 1.9–4.7% of items; PPO from scratch spends the full budget, warm-started policies about as much as the rules (0.18–0.21 s at $\beta \geq 0.2$ s).

	Cap $\delta = 5\%$				Cap $\delta = 2.5\%$			
Method	0.1 s	0.15 s	0.2 s	0.3 s	0.1 s	0.15 s	0.2 s	0.3 s
Classical								
ResNet only	129.2	129.2	129.2	129.2	129.2	129.2	129.2	129.2
Greedy cascade with defer	117.8	109.5	108.8	96.9	121.0	114.1	109.6	99.1
Paced cascade, tuned on validation	108.8	106.1	106.0	100.5	112.9	113.4	108.2	97.2
Deep RL (PPO), from scratch								
Central	122.8 ± 3.1	123.0 ± 2.4	120.7 ± 2.5	114.8 ± 1.8	122.1 ± 1.6	120.3 ± 3.0	121.4 ± 0.8	115.4 ± 0.8
FedAvg	121.6 ± 3.0	120.4 ± 3.5	118.7 ± 3.3	116.9 ± 4.2	129.3 ± 0.4	126.8 ± 1.3	126.6 ± 0.5	122.1 ± 0.8
Deep RL (PPO), warm-started from the paced rule								
Imitation only	112.3	107.6	104.0	97.9	114.5	115.8	109.8	99.8
Central, free PPO	112.5 ± 2.0	109.5 ± 2.1	107.0 ± 2.0	99.7 ± 1.4	117.1 ± 3.2	114.7 ± 2.2	112.1 ± 2.7	102.9 ± 1.2
FedAvg, free PPO	117.1 ± 2.3	111.8 ± 1.6	107.5 ± 1.7	101.0 ± 1.2	118.0 ± 2.1	113.5 ± 1.1	112.3 ± 1.1	103.9 ± 1.5
Central, anchored PPO	111.5 ± 0.7	105.3 ± 0.5	102.8 ± 1.3	97.4 ± 1.6	113.7 ± 0.5	116.0 ± 0.9	107.6 ± 1.4	98.8 ± 1.0
FedAvg, anchored PPO	112.7 ± 1.6	106.5 ± 0.5	102.9 ± 0.8	98.3 ± 1.8	114.9 ± 0.9	115.2 ± 0.5	109.3 ± 0.5	99.2 ± 0.6

in four. FedAvg matches central training throughout, with 7,621-parameter updates. We chose the anchored variant after seeing the test results, so even the one winning cell is optimistic.

9. Latency and uplink against prior measurements

Table 9 reports the time spent per item as a metric for the main methods, next to the Jetson measurements of Farahani et al. [2] and Azimi et al. [3] for SmolVLM-256M, the onboard model both papers deploy.

The units differ, and the comparison is indicative only. An EVLM segment covers several frames and generates a textual report; our item is one component crop answered by scoring options. With this caveat, three points stand out. First, the best rules and the warm-started policies answer an item in 0.18–0.25 s with 1.6–3.4 kB of upload, below the 25 kB per segment of EVLM and well within the 1.16 s frame interval EVLM assumes; EVLM itself needs up to 6.45 s per 4 s segment, so a queue builds up. Second, calling every tool costs 1.81 s and 34.5 kB per item for a risk only 1–2 points below the uncertainty cascade, which shows why selective calls matter. Third, our onboard SmolVLM call (80.8 ms on the desktop GPU, 0.73 s with the factor 9) is 3.7 times faster than the measured 2.73–2.77 s onboard request of [3], which includes text generation and a full frame. If onboard calls cost that much, only the 3 s budget would fit one onboard VLM call per item, and selective calling and deferral would matter more, not less. A Jetson measurement of option scoring is needed to settle the factor.

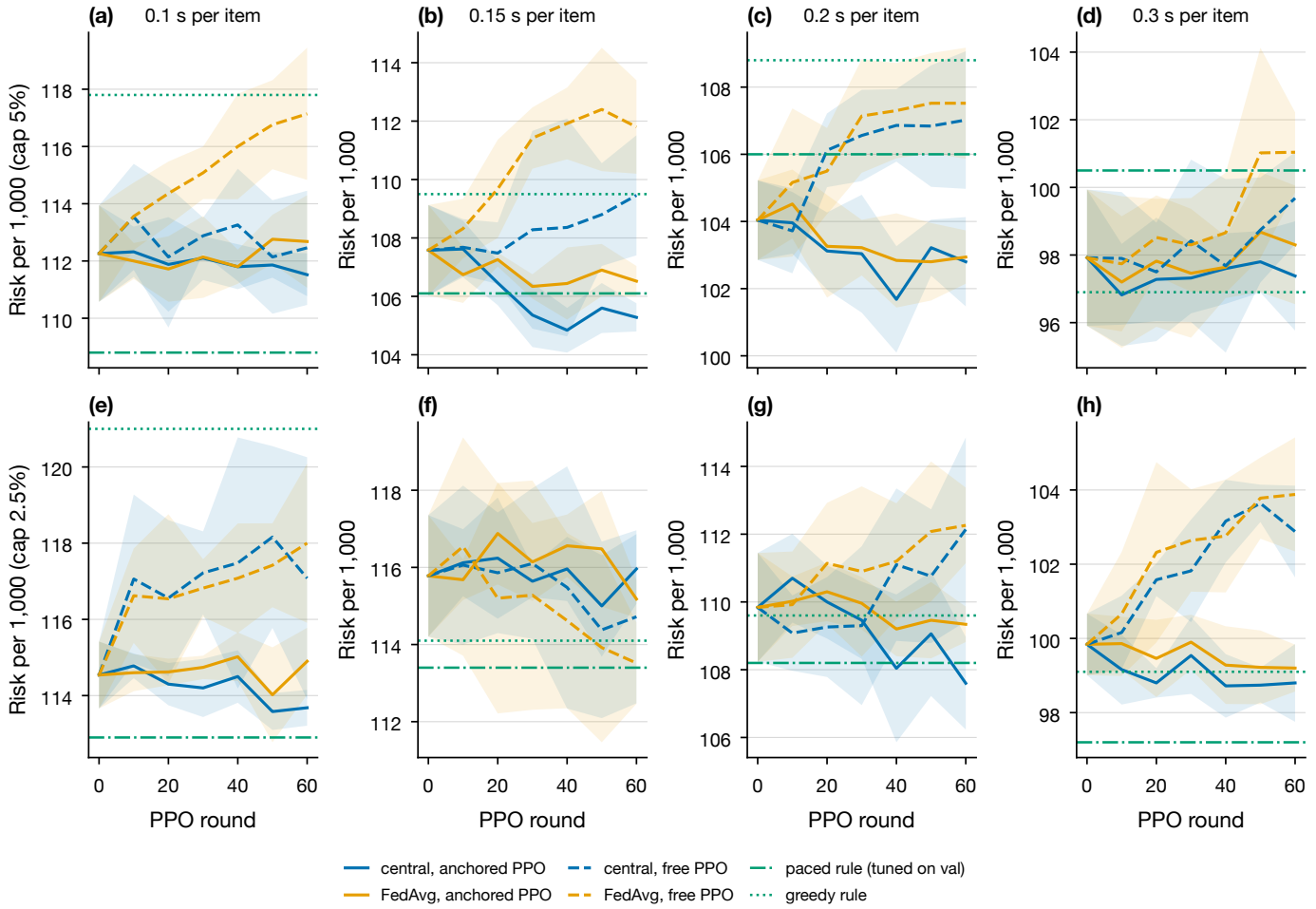


Figure 3: Warm-started PPO stays near the rule it copied; only the anchored variant holds or slightly improves it. Risk per 1,000 test items over PPO rounds; round 0 is the policy right after imitating the paced rule. Rows: deferral cap 5% (a–d) and 2.5% (e–h); columns: time budget per item. Solid: anchored PPO ($\alpha = 0.5$); dashed: free PPO. Blue: central; orange: FedAvg over 5 drones. Lines are means of 5 seeds, bands one standard deviation. Green: paced rule tuned on validation (dash-dot) and greedy rule (dotted).

10. A federated routing controller under one-sided feedback

The tool-selection experiments give the policy the true risk of every episode. In deployment, a drone sees the label of a frame only if an operator reviews it. We studied this in a mission replay with one decision per frame: after the classifier, a 29 KB multilayer perceptron (7,171 parameters) chooses to accept, ask the onboard SmolVLM, or send the frame to the cloud, subject to a 4 s deadline, an onboard queue and the weak uplink. Clients are five operator regions that keep their images and labels, built from the 18 training towers (4,579–6,449 frames each, two with a single tower).

With labels for every frame, FedAvg reaches 115.4 ± 1.6, equal to central training (114.6) and 26% below the static cascade, for 6.7–7.9 MB of total uplink (Fig. 4a, Table 10). Regions alone are no better than the cascade (157.9 ± 20.4); the two single-tower regions never learn to escalate. Unlike the tool-selection policy, this controller sees the classifier's full probability vector and the component, and chooses between tools whose errors differ strongly, which leaves room to learn.

Deployment feedback is one-sided: accepting a frame reveals nothing, the cloud answer is a noisy label (accuracy 0.79), and an operator labels a frame only when it is sent and reviewed. An audit sends accepted frames to the operator with probability λ. Learning from cloud answers makes the controller worse than the cascade (175.7), because the cloud is wrong on 1,322 test items where the classifier is right. Without audits, operator labels are too few (386 across the fleet). Only federated learning from operator labels with a 5% audit beats the cascade (142.4 ± 3.6 with inverse-propensity weights); regions alone with the same audit do not (157.3). The audit uploads (76 MB) exceed the controller updates (28.7 MB).

11. Federating perception instead

The last option is to federate the perception model and keep a fixed rule. Zero-shot, laya-vision fails on this data

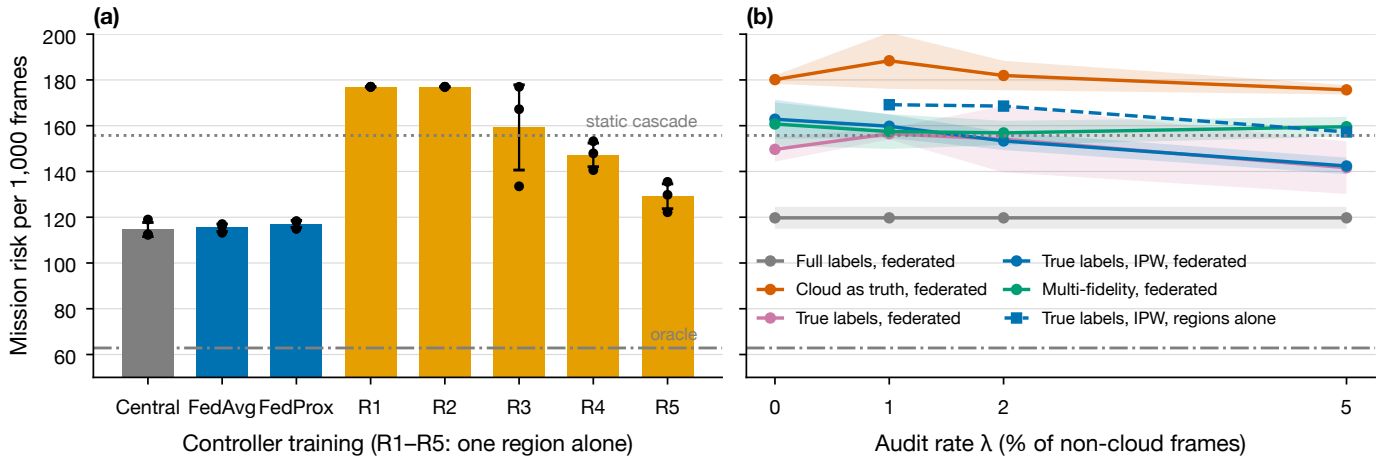


Figure 4: A federated 29 KB routing controller matches central training; under one-sided feedback only audited operator labels, pooled across regions, beat the cascade. Mission risk per 1,000 test frames, weak network. (a) Full feedback (offline): bars are means over 3 training seeds, dots the seeds, whiskers one standard deviation. R1–R5 train alone. (b) Online learning over 10 rounds against the audit rate λ; bands are one standard deviation over 3 seeds. Dotted line: static cascade; dash-dotted line: oracle.

Table 9: Latency and uplink. Prior work: measured on a Jetson Orin per request or per 4 s video segment, under low to high bandwidth of the same LTE traces. This work: mean time charged per component item (onboard tools ×9 estimate, weak LTE) and mean upload per item, test towers.

System	Latency	Uplink
Prior work (SmolVLM-256M, measured)		
EVLM, intent-driven, per 4 s segment [2]	1.4–6.4 s	25 kB
Onboard, per request [3]	2.73–2.77 s	0
Cloud, per request [3]	0.77–3.87 s	frame
Split after projection, per request [3]	2.17–3.59 s	features
This work, single tool call (per item)		
SmolVLM-256M + LoRA, onboard	0.73 s	0
laya-vision, onboard	0.70 s	0
Qwen3-VL-8B, cloud (median upload 85 ms)	0.27 s	33 kB
This work, tool selection (per item, 3 s budget)		
ResNet only	0.07 s	0
All tools that fit	1.81 s	34.5 kB
Uncertainty cascade	1.22 s	0
Cascade with defer	0.25 s	3.4 kB
PPO with defer (8,192 episodes, entropy 0.05)	0.63 s	16.4 kB
This work, mission segments ($\delta = 5\%$, $\beta = 0.2$ s)		
Paced cascade	0.18 s	1.6 kB
Anchored PPO (central)	0.18 s	1.6 kB

Table 10: Routing controllers on the test towers, weak network, mean ± standard deviation over 3 training seeds (5 trace seeds each). Uploads: frames sent by audits and escalations while learning.

Controller	Risk	Labels	Uploads
Classifier only	177.0	–	–
Static cascade	155.7	–	–
Oracle (hindsight)	62.9	–	–
Full feedback, offline			
Central	114.6 ± 3.1	25,499	–
FedAvg	115.4 ± 1.6	25,499	–
Regions alone	157.9 ± 20.4	4,579–6,449	–
One-sided feedback, online, federated			
Cloud as truth, $\lambda = 5\%$	175.7 ± 2.1	2,443	141 MB
True labels, $\lambda = 0$	149.6 ± 5.4	386	30 MB
True labels, IPW, $\lambda = 5\%$	142.4 ± 3.6	1,727	76 MB
Multi-fidelity, $\lambda = 2\%$	156.8 ± 5.3	1,052	58 MB
One-sided feedback, online, regions alone			
True labels, IPW, $\lambda = 5\%$	157.3 ± 5.3	2,047	101 MB

(condition macro F1 0.180); fine-tuned centrally, it reaches macro F1 0.703 and defect AUROC 0.965 at 74 ms per image. We federated it in 42 configurations: 3 to 18 drones, the decision head, LoRA of rank 8, the last 4 or all 20 language layers, or the full model, in fp16 or int8, from the published checkpoint or from a server model fine-tuned on 13 server-held towers, with few labels per drone and with drones that see different components. Each configuration has one seed.

One full update is 115 MB in int8, about 4,000 times the 28 KB update of the tool-selection policy, and a training run sends 0.4–41 GB against about 4 MB for 30 rounds of policy training on five drones (Fig. 5, Table 11). From the server model every update lifts defect AUROC from 0.903 to 0.940–0.962, but the small updates double the calibration error (0.06 to 0.14–0.15), because each fleet drone's images hold 34–67% defects while the test towers hold 10%. As in the fusion, the defect share differs between where a model is trained and where it is used, and probabilities must be recalibrated before a threshold can use them.

12. Discussion

What a fleet should learn. Across all options, the largest gains came from classical steps: fusing the tools' answers and correcting the threshold for the deployment's defect share (177.0 to 107.1), then deferring uncertain items to

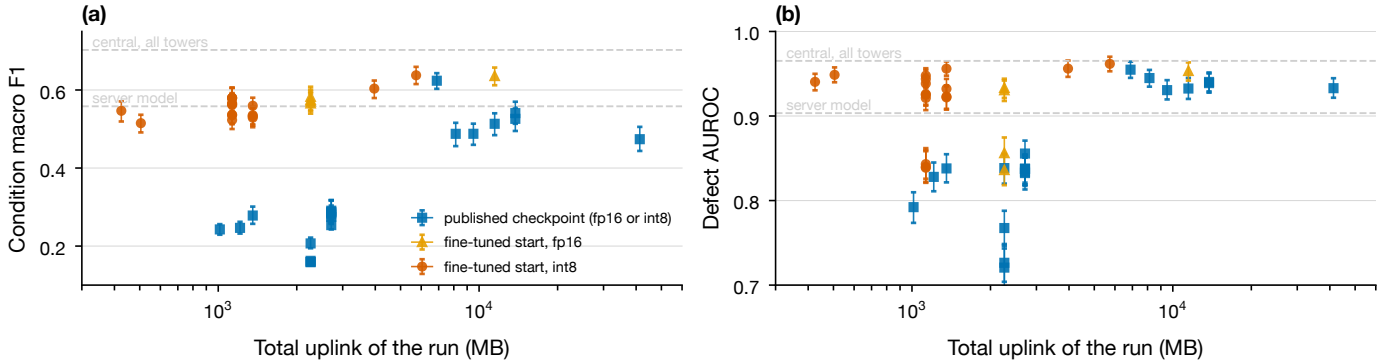


Figure 5: Federating the perception model costs gigabytes, and the start matters more than the uplink. Test condition macro F1 (a) and defect AUROC (b) of laya-vision for 42 federated configurations against the total uplink of the run, by start. Whiskers are 95% bootstrap intervals; dashed lines mark central training and the server model. One seed per run.

Table 11: Federated laya-vision from the server model (5 fleet drones, 10 rounds, int8 updates, one seed). F1: condition macro F1. ECE of the defect probability, raw and after a Platt fit on 1,500 validation images.

Update	Uplink	F1	AUROC	ECE	Platt
Server model	0	0.558	0.903	0.063	0.027
Head	425 MB	0.547	0.940	0.149	0.027
LoRA r8	505 MB	0.515	0.949	0.143	0.024
Last 4 layers	1,130 MB	0.537	0.948	0.144	0.025
20 layers	3,965 MB	0.604	0.956	0.085	0.016
Full	5,735 MB	0.638	0.962	0.065	0.021

the operator (to 92–94). Both need no reinforcement learning, and the EM estimate needs only a mean over posteriors from the fleet. A routing controller that sees the classifier's full output gains 40 risk points over a static cascade for about 7 MB when labels are available. Deep RL policies for tool selection save time against calling every tool, and federated training matches central training at 28–30 KB per update, but they match tuned rules rather than beat them. Federating perception improves defect ranking for gigabytes and needs recalibration. Using a VLM as the agent that reads tool outputs made reports worse in every configuration we tried.

Why rules are hard to beat here. With a separate budget per item, the expected risk of reporting, $\rho(q_C)$, is nearly a sufficient statistic for the next action, and a threshold on it is what the cascades implement. The label oracle's lead comes from items at the threshold where only the label tells which tool subset is right, so no policy without labels can reach it. Shared constraints over a segment add a planning problem, and there the paced rule already captures most of it; PPO from scratch could not learn it with the samples we could afford, and with a warm start it stays near the rule. Reinforcement learning may pay off when the tools are more complementary per item, for example tools that see different resolutions, or when constraints interact over longer horizons than 40 items.

Limitations. Tool answers are cached and the fleet, the network and the onboard times are simulated; the Jetson factor of 9 is an estimate, and the risk weights are assumptions. The tool-selection experiments use 3–5 seeds, the agentic and perception experiments one seed. The anchored variant of warm-started PPO was chosen after seeing test results. The tool-selection report covers the defect decision only, not the component and condition questions. FedAvg for tool selection is computed in process; it equals FedAvg in Flower, which we used for the agentic experiments. The fusion is fitted with labels from validation towers, which a deployed fleet would need to collect. All data come from one benchmark and one asset type.

13. Conclusion

We asked what an inspection drone fleet should learn to use its perception tools well. A VLM that reads its tools' outputs got worse with tools, with and without PPO and FedAvg. Classical steps gave the gains: fusing the tools' answers with a label-free threshold lowered the risk from 177.0 to 107.1 per 1,000 items, and deferring uncertain items lowered it to 92–94 at 0.25 s per item. Deep RL learned budget-aware tool selection, centrally and federated with 28–30 KB updates, and with a warm start from a tuned rule it matched that rule under shared time and deferral constraints, but it beat it clearly in only one of eight settings. For tool selection on these tools, a fleet should federate the fusion and its threshold statistics, defer at the threshold, and treat a learned policy as a cheap way to match, not replace, a tuned rule.

Data and code availability

The code, the cached tool answers and the result files are in the project repository, branches vision-fl and agentic-ppo-v2 (to be made public on acceptance). ElecVQA-Bench and InsPLAD are public [20, 21]. The laya-vision weights are published under CC BY-NC-SA 4.0 [6].

Declaration of competing interest

The author declares no competing interests.

CRediT authorship contribution statement

Mahdi Khashan: Conceptualization, Methodology, Software, Investigation, Formal analysis, Writing – original draft, Writing – review & editing.

Declaration of generative AI use

During the preparation of this work the author used an AI assistant to write code, analyse results and draft text. The author reviewed and edited the content and takes full responsibility for the publication.

Acknowledgments

Experiments in this work were run and tracked with OpenResearch [25].

References

[1] R. Farahani, N. Hossein Motlagh, Z. Azimi, C. Timmerer, L. Carnevale, S. Tarkoma, S. Dustdar, From pixels to semantics: Edge AI for UAV-based critical infrastructure inspection, arXiv:2609.18448 (2026).

[2] R. Farahani, Z. Azimi, I. Murturi, A. Goknil, S. Sen, C. Timmerer, S. Dustdar, EVLM: Intent-driven edge vision language model for UAV-based power line inspection, in: 2026 IEEE International Conference on Edge Computing and Communications (EDGE), 2026. https://doi.org/10.1109/EDGE72783.2026.00014.

[3] Z. Azimi, R. Farahani, S. Dustdar, C. Timmerer, Cloud, edge, or split? Profiling onboard and split vision-language model deployment for drone AI, arXiv:2609.25415 (2026).

[4] A. Marafioti, et al., SmolVLM: Redefining small and efficient multimodal models, arXiv:2504.05299 (2025).

[5] Convai Innovations, Laya, software (2026). https://github.com/NandhaKishorM/laya.

[6] Laya Vision, software, commit 9e1e241 (2026). https://github.com/r33drichards/laya-vision; weights https://huggingface.co/thaitea/laya-vision.

[7] Sabanovic, Maliakel, Brandic, INAR-VL: Input-aware routing for edge-cloud vision-language inference, arXiv:2605.18853 (2026).

[8] H. Zhang, S. Cha, H. B. Beytur, K. S. Chan, G. de Veciana, H. Vikalo, Online learning for multi-layer hierarchical inference under partial and policy-dependent feedback, arXiv:2603.04247 (2026).

[9] W. Wei, T. Yang, H. Chen, Y. Zhao, F. Dernoncourt, R. A. Rossi, H. Eldardiry, Learning to route LLMs from bandit feedback: One policy, many trade-offs, arXiv:2510.07429 (2025).

[10] A. Agarwal, H. B. McMahan, Z. Xu, An empirical evaluation of federated contextual bandit algorithms, arXiv:2303.10218 (2023).

[11] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, arXiv:1707.06347 (2017).

[12] J. Schulman, P. Moritz, S. Levine, M. Jordan, P. Abbeel, High-dimensional continuous control using generalized advantage estimation, in: International Conference on Learning Representations (ICLR), 2016.

[13] M. Saerens, P. Latinne, C. Decaestecker, Adjusting the outputs of a classifier to new a priori probabilities: A simple procedure, Neural Computation 14 (1) (2002) 21–41.

[14] D. J. Beutel, et al., Flower: A friendly federated learning research framework, arXiv:2007.14390 (2020).

[15] B. McMahan, E. Moore, D. Ramage, S. Hampson, B. A. y Arcas, Communication-efficient learning of deep networks from decentralized data, in: Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS), 2017.

[16] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, V. Chandra, Federated learning with non-IID data, arXiv:1806.00582 (2018).

[17] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, V. Smith, Federated optimization in heterogeneous networks, in: Proceedings of Machine Learning and Systems (MLSys), 2020.

[18] Askin, et al., Federate the router: Learning language model routers with sparse and decentralized evaluations, arXiv:2601.22318 (2026).

[19] Solat, Lee, Seif, Niyato, Poor, FedHLM, IEEE Internet of Things Journal (2025). https://doi.org/10.1109/jiot.2025.3618095.

[20] ElecVQA-Bench, dataset and code repository (2026). https://github.com/GhostDino/elecvqa-bench.

[21] A. L. B. Vieira e Silva, et al., InsPLAD: A dataset and benchmark for power line asset inspection in UAV images, International Journal of Remote Sensing (2023). arXiv:2311.01619.

[22] J. van der Hooft, S. Petrangeli, T. Wauters, R. Huysegems, P. R. Alface, T. Bostoen, F. De Turck, HTTP/2-based adaptive streaming of HEVC video over 4G/LTE networks, IEEE Communications Letters 20 (11) (2016) 2177–2180.

[23] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.

[24] E. J. Hu, et al., LoRA: Low-rank adaptation of large language models, in: International Conference on Learning Representations (ICLR), 2022.

[25] alphaXiv, OpenResearch: The local-first harness and workspace for research agents (2026). https://github.com/alphaXiv/OpenResearch.